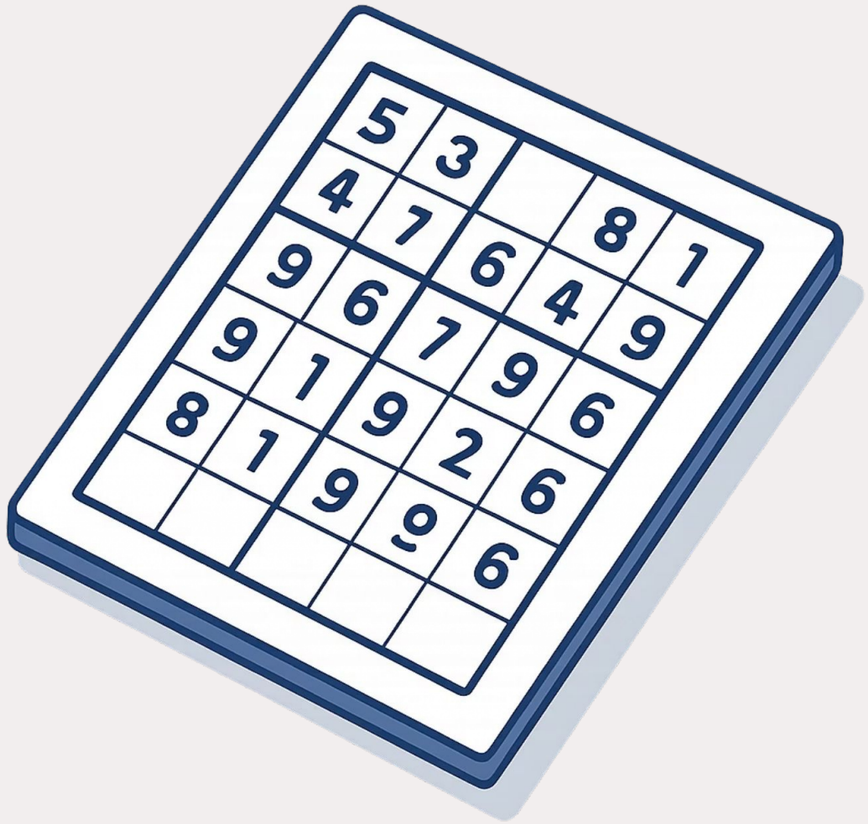
數獨遊戲專題

18組 邱岱瑋 鍾錦沛

程式設計專題

互動遊戲



專題動機與目標

🎯 學習目標

透過實作數獨遊戲，深入理解二維陣列、迴圈邏輯與條件判斷等核心程式概念。

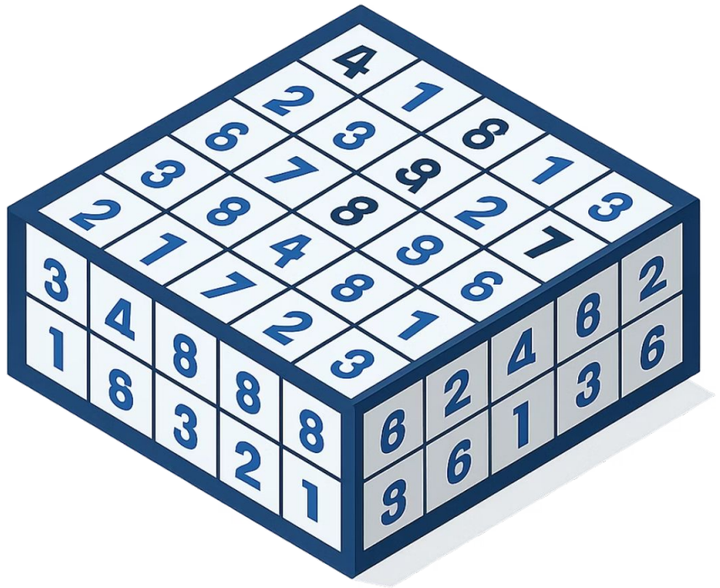
🧩 遊戲設計

設計具備規則驗證與錯誤計數的互動機制，讓遊戲不僅能玩，還能引導玩家思考。

💡 問題解決

從固定題目出發，逐步挑戰動態生成題目、難度分級等進階功能。

數獨遊戲規則簡介



基本規則

數獨是一個 9×9 的方格，玩家需在空白格中填入 1～9 的數字，並遵守以下三項規則：

- 每一**列**的數字不重複
- 每一**行**的數字不重複
- 每個**3×3 宮格**的數字不重複

核心功能一覽

固定題目顯示

遊戲啟動時載入預先設計好的數獨題目，部分格子已填入數字作為提示。

玩家數字輸入

玩家可在空白格中輸入 1～9 的數字，系統即時接收並儲存輸入內容。

答案正確性驗證

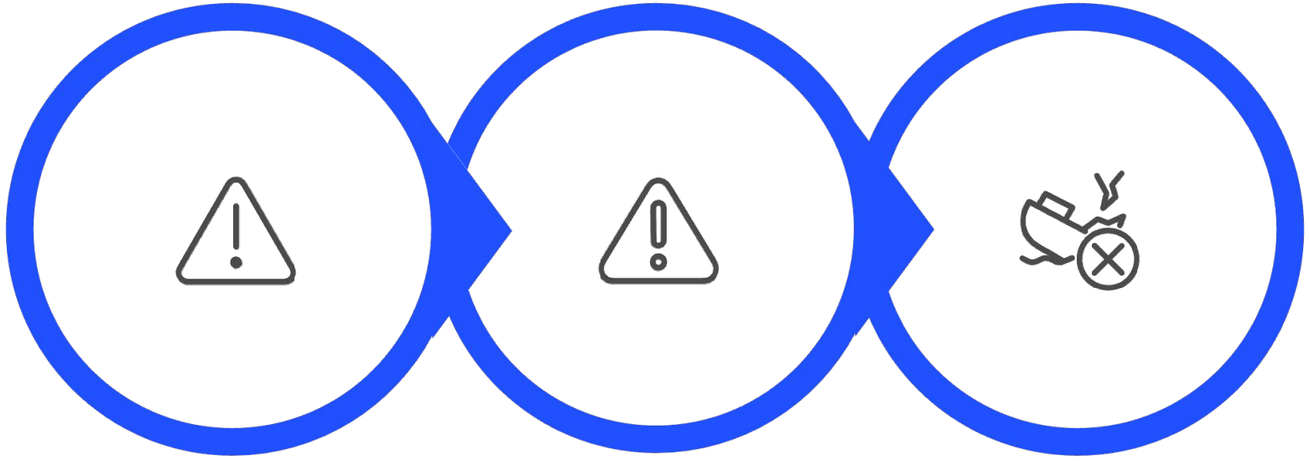
系統自動比對玩家輸入是否符合數獨規則，即時回饋正確或錯誤。

三次錯誤限制

累計三次錯誤輸入後，遊戲結束並顯示「遊戲失敗」訊息，增加挑戰性。

錯誤限制機制設計

本遊戲的核心特色在於**三次錯誤即結束** 的機制，讓玩家在每次輸入前需更審慎思考。



第一次錯誤　　第二次錯誤　　第三次錯誤

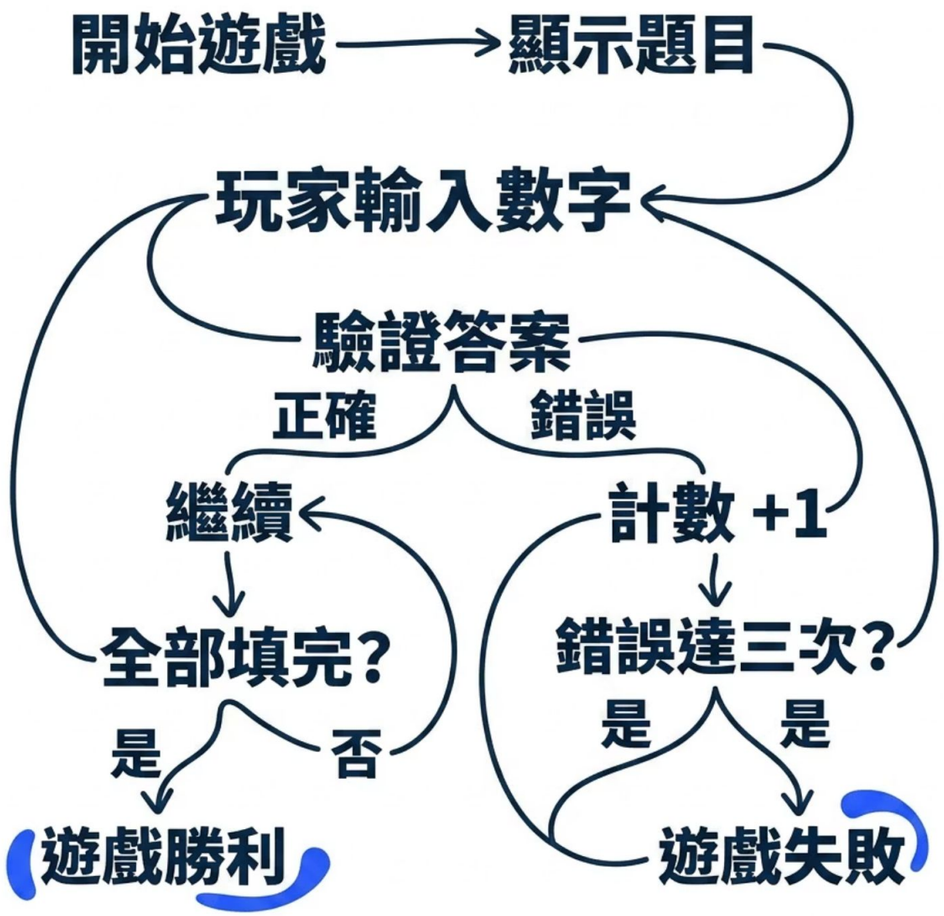
此機制不僅增加遊戲的緊張感，也訓練玩家的邏輯推理能力與細心程度。

```
bool Sudoku::validateAndSet(int r, int c, int val) {
    // 比對玩家輸入的 val 是否等於預先存好的正確答案 solution[r][c]
    if (solution[r][c] == val) {
        grid[r][c] = val; // 只有正確時才會更新玩家看到的盤面
        return true;
    }
    return false; // 錯誤則回傳 false，且不更新 grid
}
```

機制：程式在初始化時會準備兩張表：`solution`（完整正確答案）和 `grid`（玩家看到的題目）。

動作：當玩家輸入數字時，它會直接去 `solution` 的對應座標看答案是不是這個。如果不是，`grid` 就不會被改變，數字也就填不上去。

遊戲流程架構



流程說明

遊戲從載入固定題目開始，玩家逐步填入數字。每次輸入後系統即時驗證，正確則繼續，錯誤則累計計數。

> 當錯誤次數達到 **3 次**，遊戲立即結束並顯示失敗訊息，不再接受輸入。

技術實作重點

二維陣列儲存題目

使用 9×9 二維陣列儲存數獨題目，以 `0` 代表空白格，其他數字代表預填提示。

錯誤計數變數

設定整數變數記錄錯誤次數，每次錯誤後遞增，並以條件判斷是否達到上限。

輸入驗證函式

撰寫獨立函式檢查玩家輸入是否符合列、行、宮格三項規則，回傳布林值結果。

目前進度與成果

1

固定題目

已完成預設題目的設計與載入功能

3

錯誤機會

實作三次錯誤限制與遊戲結束機制

9×9

遊戲棋盤

完整呈現標準數獨方格介面

未來擴展方向

進階功能規劃

01

動態題目生成：每次遊戲隨機產生不同題目，提升可重複遊玩性

02

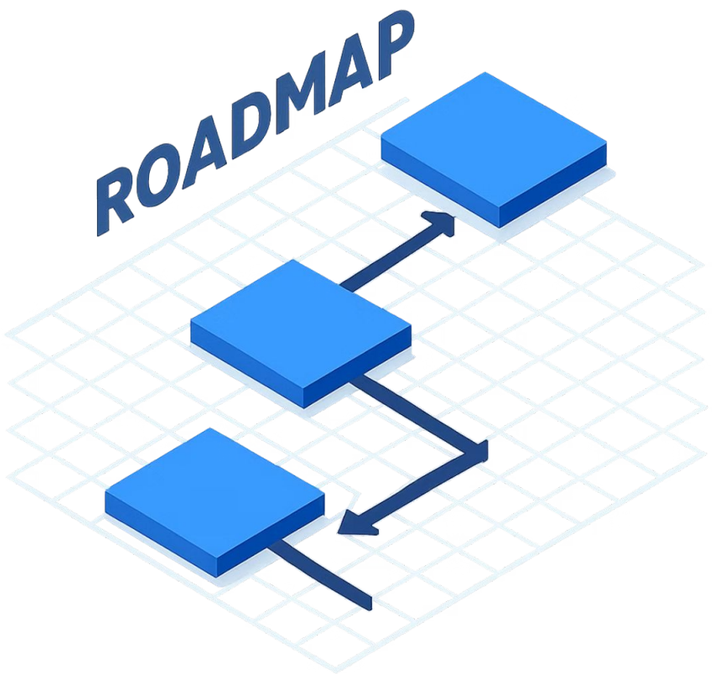
難度分級：加入簡單、中等、困難三種難度，依空白格數量區分

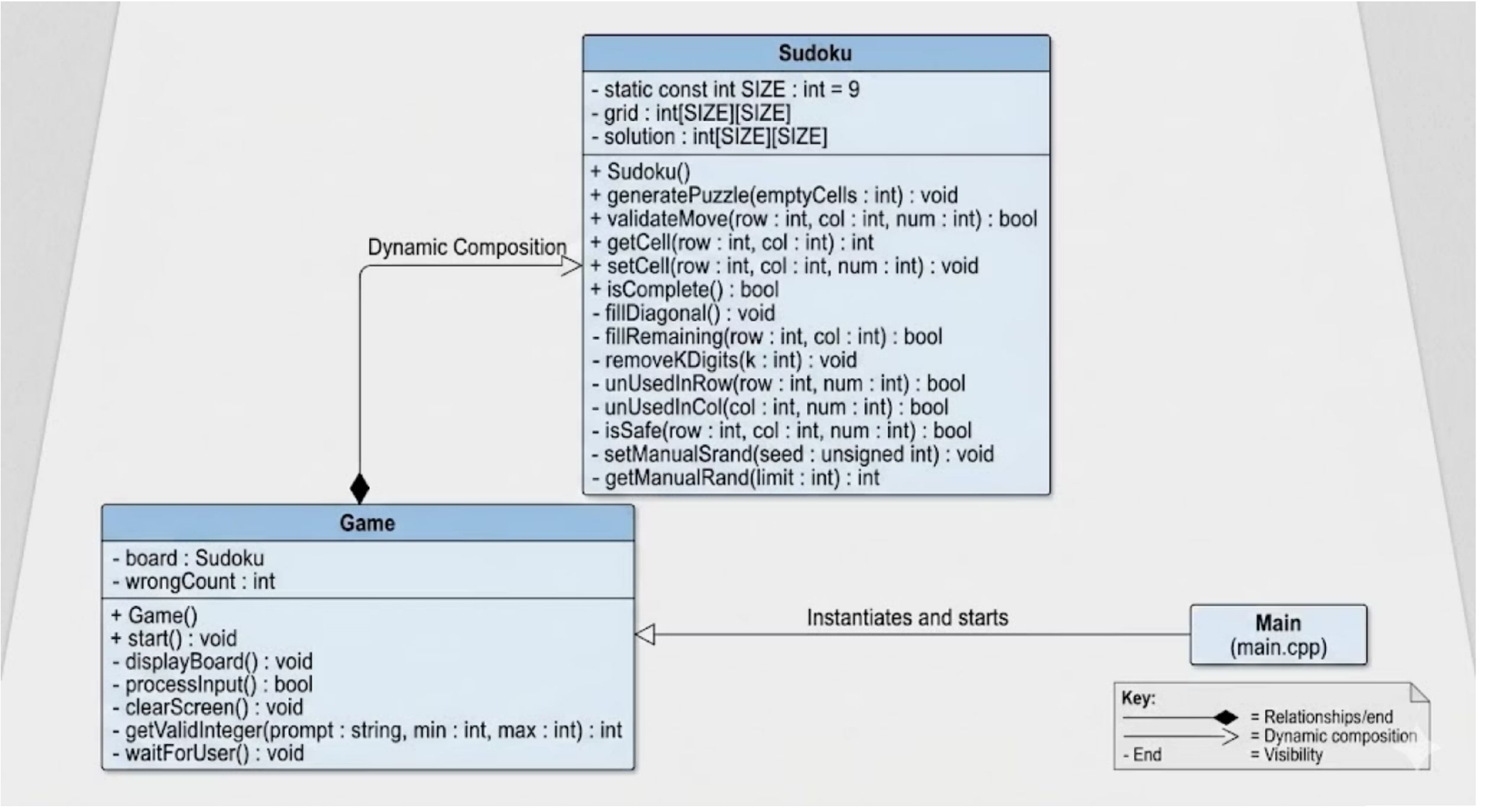
03

計時功能：記錄玩家解題時間，增加競速挑戰元素

04

提示系統：提供有限次數的提示，協助玩家突破困境




Sudoku
- static const int SIZE : int = 9
- grid : int[SIZE][SIZE]
- solution : int[SIZE][SIZE]
+ Sudoku()
+ generatePuzzle(emptyCells : int) : void
+ validateMove(row : int, col : int, num : int) : bool
+ getCell(row : int, col : int) : int
+ setCell(row : int, col : int, num : int) : void
+ isComplete() : bool
- fillDiagonal() : void
- fillRemaining(row : int, col : int) : bool
- removeKDigits(k : int) : void
- unUsedInRow(row : int, num : int) : bool
- unUsedInCol(col : int, num : int) : bool
- isSafe(row : int, col : int, num : int) : bool
- setManualSrand(seed : unsigned int) : void
- getManualRand(limit : int) : int
Dynamic Composition
Game
- board : Sudoku
- wrongCount : int
+ Game()
+ start() : void
- displayBoard() : void
- processInput() : bool
- clearScreen() : void
- getValidInteger(prompt : string, min : int, max : int) : int
- waitForUser() : void
Instantiates and starts
Main
(main.cpp)
Key:
= Relationships/end
= Dynamic composition
- End
= Visibility

謝謝大家

github:https://github.com/weiii0331/-18#